

WChain Hash Function

Keting Jia^{*}, Guoxiao Liu, Qingliang Hou, Zhuoqing Peng,
Yantian Shen, Xiaoen Lin, and Zhengrong Lu

^{*}Corresponding submitter: Keting Jia
Tsinghua University, Beijing, P.R. China
`ktjia@tsinghua.edu.cn`

Tsinghua University, Beijing, P.R. China
Zhongguancun Laboratory, Beijing, P.R. China
Shandong University, Qingdao, P.R. China
State Key Laboratory of Cryptography and
Digital Economy Security, P.R. China

Contents

1	Introduction	4
1.1	Contribution	4
2	Notation and Parameter Sets	6
2.1	Notation	6
2.2	Parameter Sets	6
3	Specification	7
3.1	Message Padding	7
3.2	FChain Structure	7
3.3	Compression Function	9
3.4	Round Function	10
3.4.1	State Representation	10
3.4.2	Round-Constant Layer	10
3.4.3	S-box Layer	11
3.4.4	Intra-column Permutation π	11
3.4.5	Mix Words Layer	12
3.5	Message Expansion	13
4	Design Rationale	14
4.1	FChain Structure	14
4.2	Component Design	14
4.3	Permutation Design	16
4.4	Features	16
5	Security Analysis	17
5.1	Generic Structural Attacks on FChain	18
5.1.1	Length extension attack.	18
5.1.2	Multicollisions and checksum correction.	19
5.1.3	Herding attacks.	20
5.1.4	Long-message second preimages.	20
5.2	Component Cryptanalysis	21
5.2.1	S-box Properties	21
5.2.2	Column Permutation $F_\pi = \pi \circ S$	21

5.2.3	Mix Words Layer Cryptanalysis	21
5.2.4	Full-Diffusion Check of the Permutation Core	23
5.3	Differential and Linear Characteristics of Permutation	24
5.4	Meet-in-the-Middle Preimage Attack on the Compression Function	27
5.5	Zero-Sum Attacks	28
5.6	Round Constants and Invariant Structures	29
6	Implementation and Performance	29
6.1	Reference Implementation	29
6.2	Optimized Scalar Implementation	30
6.3	SIMD Batch Implementation	30
6.4	Version I 18-Round Performance Measurements	31
6.5	Version II Implementation Status	32
6.6	Implementation Summary	32
7	Statistical Randomness Tests	33

1 Introduction

Hash functions are foundational cryptographic primitives for data integrity. They are also used in digital signatures, message authentication, blockchains, post-quantum public-key cryptosystems, and many higher-level protocols. China’s next-generation commercial cryptography program has issued requirements for new hash functions and public-key algorithms. In this setting, next-generation hash functions are expected to support digest sizes of 512 bits and potentially 1024 bits. These requirements motivate the design of high-security hash functions that remain efficient in software and hardware. Hash functions not only provide integrity as standalone primitives, but they are also indispensable components of post-quantum public-key systems, where they can directly affect the performance of the full scheme. This document specifies and analyzes the proposed high-security hash family **WChain**.

Iterative structures play a crucial role in the design of hash functions. Since its introduction in the 1980s, the Merkle-Damgård (MD) construction has served as the classic iterative paradigm for hash functions, forming the foundational architecture for a series of international standards, including MD5, SHA-1, SHA-2, and SM3. In 2004–2005, Wang et al. introduced modular differential cryptanalysis and mounted practical collision attacks against MD5 and SHA-1. This breakthrough ignited a global surge in cryptographic hash-function research. Furthermore, inherent structural weaknesses in the MD construction render it susceptible to length-extension attacks, multicollision attacks, and second-preimage attacks on long messages. During the development of the SHA-3 hash standard, the United States explicitly required non-MD structures. Consequently, several hash constructions emerged, including double-pipe, wide-pipe, HAIFA, Sponge, and JH structures. Keccak, built on the Sponge construction, was ultimately selected as SHA-3. Introduced by Bertoni et al. in 2007, the Sponge construction supports fixed-length digests and extendable-output functions (XOFs), and it can also be used as a basis for stream-cipher modes, Authenticated Encryption with Associated Data (AEAD), Key Derivation Functions (KDFs), and Pseudorandom Functions (PRFs). Bertoni et al. proved the security of Sponge in the ideal-permutation model under the indistinguishability framework. At CRYPTO 2022, Lefevre and Mennink established tighter preimage security bounds for Sponge. To improve compression efficiency, several Sponge variants have been proposed, including the feed-forward Sponge-F variant introduced by Chun Guo et al. and the Sponge-DM construction proposed by Siwei Sun et al. In the ideal-permutation model, these structures provide rigorous security arguments for preimage, second-preimage, and collision resistance.

1.1 Contribution

This document specifies the **WChain** hash family. The design uses an **FChain**-style feed-forward compression structure, a final checksum block, and a word-oriented nonlinear permutation built from a 9-bit NAND2/chi S-box, a fixed intra-column permutation π , and a

sparse five-input Mix Words layer L_{5w} . The goal is to keep the implementation close to ordinary 64-bit word operations while increasing the structural constraints faced by classical Merkle–Damgård-style generic attacks.

Cryptographic hash functions are basic primitives for digital signatures, message authentication, key derivation, and data-integrity verification. WChain uses an iterative feed-forward compression function. In each compression call, the current chaining value is denoted by V , the current message block is denoted by B , and the internal state transformation is driven by message-injection blocks expanded from B .

WChain keeps the feed-forward compression structure and the Feistel-type message expansion, and uses a 5-word cyclic-block layer L_{5w} as its main linear diffusion component. The layer is lightweight: each output bit depends on five input bits. Its rotation table was selected to keep the matrix full rank, preserve low-weight branch-number targets in both differential and transpose-linear directions, avoid the detected low-weight iterative structures, and make every inverse single-bit dependency cover all 64 z columns. A lane-wise rotated round-constant layer is also included before the S-box layer to break round symmetry, slide structures, natural invariant subspaces, and generalized nonlinear invariant structures.

The algorithm has two versions:

- **Version I:** digest length $h = 512$ bits, internal state length $b = 576$ bits, state shape $3 \times 3 \times 64$, fully specified in this document;
- **Version II:** digest length $h = 1024$ bits, internal state length $b = 1152$ bits, split state shape $(9 \times 64)_L \parallel (9 \times 64)_R$, fully specified with an asymmetric right-half L_{5w} table in this document.

2 Notation and Parameter Sets

2.1 Notation

Notation	Meaning
h	Final digest length in bits
λ	Basic message-word length in bits
$b = h + \lambda$	Internal chaining-state length in bits
μ	Message-block length in bits for one compression call
$ M $	Bit length of the original unpadded message
$M_1, \dots, M_l$	Padded message blocks, each of μ bits
Σ	XOR accumulator of all padded message blocks $M_1, \dots, M_l$
IV	Initial value, $IV \in \{0, 1\}^b$
H_i	Chaining value after the i -th compression call
Z	Final h -bit digest
CF	Feed-forward compression function
$F_B(\cdot)$	Message-dependent internal state transformation driven by block B
ME	Message-expansion procedure
$\hat{B}_j$	Expanded message-injection block for round j
Round $_j$	Internal round function in round j
S	9-bit NAND2/chi S-box
π	Fixed 9-bit intra-column permutation
L_{5w}	New 5-word cyclic-block linear diffusion layer
L_{5w}^{II}	Split Version II linear layer with cross-swap and asymmetric halves
lota $_j$	Round-constant injection layer
$\pi_j^{(16)}$	The j -th 16-bit word of the hexadecimal fractional part of π
d_i	Lane-dependent rotation offset for the round constants; $d_i = i$
N	Lane length; $N = 64$ for Version I and for each half of Version II
$\oplus$	Bitwise XOR
$\parallel$	Bit-string concatenation

2.2 Parameter Sets

Table 1: WChain parameter sets

Version	Digest h	State b	Block μ	Rounds R
Version I	512	576	1152	18
Version II	1024	1152	2304	24

Both versions use the same 9-bit S-box, the same π permutation, and the same lane-wise rotated round-constant strategy. Version I applies one 9×64 permutation. Version II applies the same nonlinear layer independently to two 9×64 halves, then uses a fixed

five-pair cross-swap followed by asymmetric left/right L_{5w} layers. The Version II choice keeps the 64-bit lane implementation of Version I while doubling the state size.

3 Specification

3.1 Message Padding

The input message is first padded by a simple 10* padding rule and then parsed into message blocks

$$M_1, M_2, \dots, M_l. \quad (1)$$

Let M be the original message and let $|M|$ denote its length in bits. Padding appends one bit “1” followed by the minimum number of zero bits needed to reach a multiple of the WChain message-block length μ . No encoded message-length field is appended. Thus, for both versions,

$$M^* = M \parallel 1 \parallel 0^k, \quad (2)$$

where k is the smallest nonnegative integer satisfying

$$|M| + 1 + k \equiv 0 \pmod{\mu}. \quad (3)$$

For Version I, $\mu = 1152$; for Version II, $\mu = 2304$. The delimiter bit always adds at least one bit, so even an already block-aligned message receives a new padding suffix. The resulting encoding is reversible for bit strings: when decoding a padded string, remove trailing zero bits up to the last delimiter bit “1”. The supported maximum input length is an implementation parameter; the padding rule itself does not encode this length. In both versions, M^* is split into μ -bit blocks $M_1, \dots, M_l$.

3.2 FChain Structure

The FChain structure is designed as an enhanced Merkle–Damgård-style iteration. The classical Merkle–Damgård paradigm is attractive because it is simple, modular, and lets a fixed-input-length compression function process arbitrary-length messages. This modularity is also the reason why many standardized hash functions, including SHA-2 and SM3, can be described around an iterated compression-function interface. However, the plain form exposes well-known generic weaknesses, such as direct length extension when the chaining value is revealed, Joux multicollisions, herding attacks, and long-message second-preimage attacks.

FChain keeps the same high-level compression-function interface but changes the chaining rule and the finalization rule. Instead of publishing or directly continuing from the last ordinary Merkle–Damgård chaining value, the real message blocks are processed

with the two-step feedback

$$H_i = \text{CF}(H_{i-1}, M_i) \oplus H_{i-2}, \quad (4)$$

and all message blocks are accumulated into a final checksum block Σ . The published digest is obtained only after the final compression call with Σ and after truncation. Consequently, a generic final collision has to satisfy both an internal-state condition and a checksum condition, as formalized later in (42). This should be understood as adding constraints and raising the generic adaptation cost of classical MD attacks, not as a stand-alone proof that all MD-based attacks are impossible.

The same wrapper idea can also be applied to existing MD or HAIFA-like compression functions. For example, an SM3-like or SHA-2-like compression function could be used as the CF component inside an FChain-style iteration in order to add hidden wide-state processing, the two-step feedback, the final XOR-checksum block, and non-direct length-extension behavior without redesigning the internal compression function from scratch. Such a derived mode would not be output-compatible with the original standard hash function: it must define its own IV, padding, block mapping, digest truncation, and security parameters. The corresponding generic structural effects are discussed in Section 5.

The construction adds a final checksum block over the padded message blocks

$$\Sigma = M_1 \oplus M_2 \oplus \dots \oplus M_l. \quad (5)$$

This block binds all padded message blocks through an XOR accumulator and separates the finalization step from the last ordinary message-block compression call. The final block Σ itself is not included in the accumulator.

The initial chaining values are

$$H_{-1} = 0, \quad H_0 = IV. \quad (6)$$

For real message blocks, the iteration is

$$H_i = \text{CF}(H_{i-1}, M_i) \oplus H_{i-2}, \quad i = 1, 2, \dots, l. \quad (7)$$

The final Σ block is processed without the external two-step feedback:

$$H_{l+1} = \text{CF}(H_l, \Sigma). \quad (8)$$

The digest is

$$Z = \text{Trunc}_h(H_{l+1}). \quad (9)$$

Serialization and truncation. For both versions, a state is serialized in lane order using big-endian 64-bit encoding for each lane. Version II serializes the left half first and the right half second. The function Trunc_h returns the leftmost h bits of this serialized state.



Input:	Message M
Output:	h -bit digest Z

```

1: Pad  $M$  as in Section 3.1, then parse it as  $M_1, \dots, M_l$ 
2:  $\Sigma \leftarrow M_1 \oplus M_2 \oplus \dots \oplus M_l$ 
3:  $H_{-1} \leftarrow 0, \quad H_0 \leftarrow IV$ 
4: for  $i = 1$  to  $l$  do
5:    $H_i \leftarrow \text{CF}(H_{i-1}, M_i) \oplus H_{i-2}$ 
6: end for
7:  $H_{l+1} \leftarrow \text{CF}(H_l, \Sigma)$                                      // finalization block
8:  $Z \leftarrow \text{Trunc}_h(H_{l+1})$ 
9: return  $Z$ 

```

Let $V \in \{0, 1\}^b$ be the current chaining value and let $B \in \{0, 1\}^\mu$ be the current message block. The compression function is $\mathbf{CF}(V, B)$.

Given chaining value V , set $X_0 = V$. The message expansion derives round message-injection blocks $\hat{B}_0, \dots, \hat{B}_R$ from B . The internal state transformation is

and

$$\text{CF}(V, B) = X_R \oplus \hat{B}_R \oplus V. \quad (11)$$

The feed-forward term $\oplus V$ binds the internal transformation output to the input chaining value. The compression function is shown in Algorithm 2.

Table 3: Algorithm 2: Compression Function $\text{CF}(V, B)$

Input:	Chaining value $V \in \{0, 1\}^b$, message block $B \in \{0, 1\}^\mu$
Output:	New chaining value $H \in \{0, 1\}^b$

```

1:  $(\widehat{B}_0, \widehat{B}_1, \dots, \widehat{B}_R) \leftarrow \text{ME}(B)$ 
2:  $X_0 \leftarrow V$ 
3: for  $j = 1$  to  $R$  do
4:    $X_j \leftarrow \text{Round}_j(X_{j-1} \oplus \widehat{B}_{j-1})$ 
5: end for
6:  $H \leftarrow X_R \oplus \widehat{B}_R \oplus V$ 
7: return  $H$ 

```

3.4 Round Function

For Version I, the j -th round function is

$$\text{Round}_j = L_{5w} \circ \pi \circ S \circ \text{lota}_j. \quad (12)$$

For Version II, the same component order is used on a split state:

$$\text{Round}_j^{\text{II}} = L_{5w}^{\text{II}} \circ \pi_{\text{split}} \circ S_{\text{split}} \circ \text{lota}_j. \quad (13)$$

The components are applied from right to left: first the round constants are injected, then the 9-bit S-box layer is applied, then the intra-column permutation π , and finally the version-specific linear layer.

3.4.1 State Representation

For Version I, the 576-bit state is represented as

$$A[x, y, z], \quad x, y \in \{0, 1, 2\}, \quad z \in \mathbb{Z}_{64}. \quad (14)$$

The row index is

$$i = 3x + y, \quad i \in \{0, 1, \dots, 8\}. \quad (15)$$

For each fixed z , the nine bits form the input to one 9-bit S-box. Version II is represented as two such halves,

$$A_L[i, z], \quad A_R[i, z], \quad 0 \leq i < 9, \quad z \in \mathbb{Z}_{64}. \quad (16)$$

Each half has its own 9-bit S-box column at every z , so Version II has 128 S-box columns per round.

3.4.2 Round-Constant Layer

The constant generation uses a transparent single-source constant derived from the hexadecimal fractional part of π :

$$\pi - 3 = 0.243\text{F6A}8885\text{A308D313198A2E03707344} \dots_{16}. \quad (17)$$

Split this stream into consecutive 16-bit words,

$$\pi_1^{(16)} = \text{0x243F}, \quad \pi_2^{(16)} = \text{0x6A88}, \quad \pi_3^{(16)} = \text{0x85A3}, \quad \pi_4^{(16)} = \text{0x08D3}, \dots \quad (18)$$

For Version I, $\pi_j^{(16)}$ is zero-extended to a 64-bit lane and injected into all nine lanes with lane-dependent rotations

$$d_i = i, \quad 0 \leq i < 9, \quad (19)$$

namely

$$X[i] \leftarrow X[i] \oplus \text{rotl}_{64}(\pi_j^{(16)}, d_i), \quad 0 \leq i < 9. \quad (20)$$

Here $X[i]$ denotes the i -th lane with $i = 3x + y$. The use of distinct rotations prevents the nine injected lanes from remaining identical; in the S-box view, the induced 9-bit column constants have rank 9 in every checked Version I round. Version II uses the same 64-bit constant source on both halves:

$$X_L[i] \leftarrow X_L[i] \oplus \text{rotl}_{64}(\pi_j^{(16)}, i), \quad X_R[i] \leftarrow X_R[i] \oplus \text{rotl}_{64}(\pi_j^{(16)}, i + 9), \quad (21)$$

for $0 \leq i < 9$. This keeps the constant generation lightweight while avoiding identical lane constants across the two halves.

3.4.3 S-box Layer

For each z column, apply the NAND2/chi 9-bit S-box. For input $(a_0, \dots, a_8)$ and output $(b_0, \dots, b_8)$,

$$b_i = a_i \oplus \overline{a_{i+1}} \wedge a_{i+2}, \quad 0 \leq i < 9, \quad (22)$$

where all indices are modulo 9. This S-box is bijective, has forward algebraic degree 2 and inverse algebraic degree 5, and satisfies

$$\text{DDT}_{\max} = 128, \quad \max |\text{LAT}| = 128. \quad (23)$$

Thus the maximum single-S-box differential probability is 2^{-2} , and the maximum linear correlation is 2^{-1} .

3.4.4 Intra-column Permutation π

The fixed 9-bit permutation is

$$\pi = [0, 7, 5, 3, 1, 8, 6, 4, 2]. \quad (24)$$

It is applied as

$$C[\pi(i), z] = U[i, z], \quad (25)$$

where U denotes the state after the S-box layer and C denotes the permuted state. The layer is only a wiring permutation in hardware. It is included because the verified diffusion and differential-characteristic searches are performed for the composed layer $L_{5w} \circ \pi$. The permutation changes the relative S-box coordinates before the sparse linear layer, reducing alignment with simple lane-local low-weight structures at negligible implementation cost.

3.4.5 Mix Words Layer

The Mix Words layer treats the state as nine 64-bit lanes

$$A[i], \quad 0 \leq i < 9, \quad (26)$$

where $i = 3x + y$ in the $3 \times 3 \times N$ representation. Each output lane is the XOR of five rotated input lanes:

$$L_{5w}(A)[i] = \bigoplus_{(j,a) \in R_i} \text{rotl}_N(A[j], a), \quad 0 \leq i < 9. \quad (27)$$

Here R_i is the rotation row for output lane i . The support is cyclic with offsets

$$\{0, 1, 2, 3, 7\} \pmod{9}, \quad (28)$$

so each output lane uses five input lanes and each input lane contributes to five output lanes.

For Version I, the specified rotation rows are listed in Table 4. All rotations are left rotations modulo 64.

Table 4: Version I rotation rows for L_{5w}

Output lane	Input lanes and rotations
0	$A[0] \lll 5 \oplus A[1] \lll 22 \oplus A[2] \lll 3 \oplus A[3] \lll 28 \oplus A[7] \lll 27$
1	$A[1] \lll 39 \oplus A[2] \lll 59 \oplus A[3] \lll 45 \oplus A[4] \lll 3 \oplus A[8] \lll 9$
2	$A[2] \lll 54 \oplus A[3] \lll 6 \oplus A[4] \lll 20 \oplus A[5] \lll 8 \oplus A[0] \lll 48$
3	$A[3] \lll 0 \oplus A[4] \lll 57 \oplus A[5] \lll 55 \oplus A[6] \lll 46 \oplus A[1] \lll 28$
4	$A[4] \lll 61 \oplus A[5] \lll 38 \oplus A[6] \lll 0 \oplus A[7] \lll 44 \oplus A[2] \lll 52$
5	$A[5] \lll 29 \oplus A[6] \lll 10 \oplus A[7] \lll 37 \oplus A[8] \lll 19 \oplus A[3] \lll 7$
6	$A[6] \lll 56 \oplus A[7] \lll 5 \oplus A[8] \lll 50 \oplus A[0] \lll 27 \oplus A[4] \lll 13$
7	$A[7] \lll 40 \oplus A[8] \lll 10 \oplus A[0] \lll 11 \oplus A[1] \lll 54 \oplus A[5] \lll 14$
8	$A[8] \lll 3 \oplus A[0] \lll 38 \oplus A[1] \lll 36 \oplus A[2] \lll 17 \oplus A[6] \lll 13$

Version II split Mix Words layer. Version II uses two 9×64 halves instead of one 9×128 lane array. Let

$$A = (A_L, A_R), \quad A_L[i], A_R[i] \in \mathbb{F}_2^{64}, \quad 0 \leq i < 9.$$

After S_{split} and π_{split} , a fixed cross-swap exchanges five word pairs:

$$(L[0], R[0]), (L[1], R[3]), (L[3], R[1]), (L[4], R[4]), (L[6], R[6]). \quad (29)$$

Denote the swapped state by $C = (C_L, C_R)$. The Version II linear layer is

$$L_{5w}^{\text{II}}(A) = (L_L(C_L), L_R(C_R)), \quad (30)$$

where L_L is the Version I table in Table 4. The right half has the same support, but the i -th output row adds an extra rotation offset

$$\delta = (11, 21, 19, 21, 43, 5, 3, 10, 5), \quad (31)$$

that is,

$$R_i^{(R)} = \{(j, a + \delta_i \bmod 64) : (j, a) \in R_i^{(L)}\}. \quad (32)$$

The full right-half table is given in Table 5.

Table 5: Version II right-half rotation rows

Output lane	Input lanes and rotations
0	$A[0] \lll 16 \oplus A[1] \lll 33 \oplus A[2] \lll 14 \oplus A[3] \lll 39 \oplus A[7] \lll 38$
1	$A[1] \lll 60 \oplus A[2] \lll 16 \oplus A[3] \lll 2 \oplus A[4] \lll 24 \oplus A[8] \lll 30$
2	$A[2] \lll 9 \oplus A[3] \lll 25 \oplus A[4] \lll 39 \oplus A[5] \lll 27 \oplus A[0] \lll 3$
3	$A[3] \lll 21 \oplus A[4] \lll 14 \oplus A[5] \lll 12 \oplus A[6] \lll 3 \oplus A[1] \lll 49$
4	$A[4] \lll 40 \oplus A[5] \lll 17 \oplus A[6] \lll 43 \oplus A[7] \lll 23 \oplus A[2] \lll 31$
5	$A[5] \lll 34 \oplus A[6] \lll 15 \oplus A[7] \lll 42 \oplus A[8] \lll 24 \oplus A[3] \lll 12$
6	$A[6] \lll 59 \oplus A[7] \lll 8 \oplus A[8] \lll 53 \oplus A[0] \lll 30 \oplus A[4] \lll 16$
7	$A[7] \lll 50 \oplus A[8] \lll 20 \oplus A[0] \lll 21 \oplus A[1] \lll 0 \oplus A[5] \lll 24$
8	$A[8] \lll 8 \oplus A[0] \lll 43 \oplus A[1] \lll 41 \oplus A[2] \lll 22 \oplus A[6] \lll 18$

The cross-swap is a word permutation and is an involution. The inverse of the Version II Mix Words Layer is defined by

$$(L_{5w}^{\text{II}})^{-1} = \text{CrossSwap}_5 \circ (L_L^{-1}, L_R^{-1}), \quad L_R^{-1} = L_L^{-1} \circ \Delta_R^{-1}, \quad (33)$$

where Δ_R denotes the output-lane rotation layer induced by (31).

3.5 Message Expansion

The message expansion keeps the Feistel-type recursion, but it uses a lighter update function than the full round function. The message block B is parsed as big-endian 64-bit words in lane order. The first b bits of B form the seed state $\widehat{B}_0$, and the second b bits form $\widehat{B}_1$. For Version II, the lane order is left half first and right half second. For each 9-word half, define

$$D(A)[i] = \text{rotl}_{64}(A[i], d_i), \quad 0 \leq i < 9, \quad (34)$$

where

$$d = (0, 1, 10, 20, 25, 32, 41, 46, 50). \quad (35)$$

Here $d_0 = 0$, and for $i \geq 1$ the value d_i is chosen from the interval $[7(i-1), 7i]$. The listed deterministic instance is derived from the $\pi^{(16)}$ stream so that the specification is reproducible. The lightweight message-expansion function is

$$\text{MEF}(A) = S(D(A)), \quad (36)$$

where the 9-bit S-box layer S is applied to the nine rotated words. For Version II, MEF is applied independently to the left and right 9-word halves. The expanded blocks are then computed as

$$\widehat{B}_j = \text{MEF}(\widehat{B}_{j-1}) \oplus \widehat{B}_{j-2}, \quad j = 2, 3, \dots, R. \quad (37)$$

Thus the expansion needs only $R - 1$ lightweight MEF calls after the two seed states have been derived from the message block. The full round function is reserved for the state update, while the message expansion uses only rotations and one S-box layer to reduce implementation cost.

4 Design Rationale

4.1 FChain Structure

WChain keeps the useful modular interface of Merkle–Damgård-style hashing: an arbitrary-length message is reduced through a fixed-input-length compression function. The difference is in the way the compression calls are connected and finalized. Ordinary message blocks are processed with the two-step feedback

$$H_i = \text{CF}(H_{i-1}, M_i) \oplus H_{i-2}$$

so the last ordinary chaining value is not a directly published continuation state. After all message blocks have been processed, the checksum block

$$\Sigma = M_1 \oplus \dots \oplus M_l$$

is compressed in a separate finalization step, and the digest is obtained by truncating the hidden wide state.

These four ingredients—two-step feedback, hidden wide state, checksum finalization, and digest truncation—are intended to raise the adaptation cost of direct length extension, Joux multicollisions, herding, and long-message second-preimage attacks compared with the plain MD iteration. The same idea can also be used as a structural upgrade path for existing MD-based algorithms: compression functions in the style of SHA-2, SM3, or related designs can be embedded into an FChain-like wrapper to obtain additional feedback and finalization constraints. Such a wrapper defines a new mode rather than an output-compatible variant, and therefore requires fresh parameters, IVs, padding, block parsing, and test vectors. The corresponding generic estimates are discussed in Section 5.

4.2 Component Design

The components are arranged around a simple design principle: use a compact column S-box for nonlinearity, use inexpensive coordinate wiring to avoid fixed alignment, and use a sparse word-mixing layer to spread activity quickly while remaining efficient on 64-bit

software platforms. This gives WChain a word-oriented permutation that is lighter than a dense binary matrix but still has explicit forward, transpose-linear, and inverse-diffusion checks.

The 9-bit NAND2/chi S-box is the only nonlinear primitive in the round. It is a bijective column operation, so all 64 physical z -columns in Version I and all 128 split S-box columns in Version II can be processed in parallel. Its low degree supports simple bit-sliced implementation, while its DDT and LAT bounds provide the local differential and linear costs used in the trail searches. The design deliberately keeps this nonlinear component small and uniform; the surrounding layers are then responsible for preventing activity from staying concentrated in a few word positions.

The intra-column permutation π is a diffusion-coupling layer rather than a second nonlinear layer. Its criteria are: it must be a 9-bit wiring permutation with negligible hardware cost and easy software fusion into the Mix Words expressions; it must preserve the S-box DDT, LAT, and algebraic-degree properties; it must break the direct alignment between S-box output coordinates and the five-word support of L_{5w} ; and it should improve the two-step column spread of

$$L_{5w} \circ \pi \circ L_{5w} \circ \pi(e_i)$$

for every single-bit input e_i . The selected value $\pi = [0, 7, 5, 3, 1, 8, 6, 4, 2]$ helps the composed $S/\pi/L_{5w}$ layer avoid concentrated low-weight propagation while preserving the same local S-box differential and linear bounds. The criterion is applied in both differential and transpose-linear directions.

The Mix Words Layer is the main global diffusion component. Its five-input sparse word form was chosen as a compromise between implementation cost and diffusion strength: each output word is computed by a fixed XOR-and-rotate network, but the full binary matrix is invertible, has balanced row and column weights, gives nontrivial low-weight branch numbers in both the differential and transpose-linear directions, and has a dense inverse. These checks are intended to prevent the lightweight word layer from becoming the source of low-weight iterative differential or linear cores. The Version I rotation table in Table 4 is the concrete parameter set that realizes these goals in the 9×64 state.

The remaining components are selected for transparency and low overhead. The lane-rotated $\pi_j^{(16)}$ round constants are reproducible and avoid identical constant patterns across S-box coordinates. The message expansion uses rotations and one S-box layer as a lightweight injection schedule rather than running a second full permutation. For Version II, the split construction keeps the same 64-bit software granularity while doubling the state. CrossSwap5 exchanges dependencies between the two halves, and the asymmetric right-half Mix Words table prevents simple left/right rotation relations of the form $R[i] = \text{rotl}_{64}(L[i], k)$. The resulting design keeps Version II close to two software-friendly 9×64 halves while avoiding an obvious Keccak-style self-differential alignment. The concrete rank, branch-number, inverse-coverage, diffusion, and self-differential checks supporting these choices are reported as component cryptanalysis in Section 5.2.

4.3 Permutation Design

The permutation follows the round structure

$$\text{Round}_j = L_{5w} \circ \pi \circ S \circ \text{lota}_j.$$

The order is chosen to inject constants before nonlinearity, apply one 9-bit S-box to every z -column, change the coordinate alignment by π , and then spread the result through the Mix Words layer. The design is intended to keep the critical path short: the S-box layer is bit-sliced across all columns, π is only wiring, and L_{5w} is a fixed XOR-and-rotate network. Thus the permutation has low implementation latency and maps naturally to scalar 64-bit software, SIMD batching, and hardware datapaths.

The permutation is specified in two sizes. Version I is a compact 576-bit permutation over a 9×64 state with 64 parallel S-box columns. Version II is a 1152-bit split permutation over two 9×64 halves. It keeps the same 64-bit word granularity while using CrossSwap5 and an asymmetric right-half Mix Words table to exchange dependencies between the halves and to break the left/right self-differential symmetry observed in the symmetric split construction.

The diffusion evidence supports a strong avalanche effect. In the tested single-bit support propagation model, active S-box columns cover the whole column set after 3 rounds, and the reachable support covers the full state after 4 rounds. The inverse Mix Words matrices are dense and give strong backward diffusion as well, so the permutation was checked in both the forward differential direction and the transpose/backward linear direction. The reduced-round trail searches give explicit differential bounds: for example, the best found 5-round differential characteristics have probabilities 2^{-520} for Version I and 2^{-768} for Version II, and the 7-round single-trail exponents are 1174 and 2087, respectively. These are found single-characteristic bounds rather than full differential-hull proofs, but they show that the sparse implementation-oriented layer still gives high reduced-round diffusion pressure.

Because the core is a standalone 576-bit or 1152-bit permutation, it can also be viewed as a reusable permutation primitive. Besides the FChain mode specified in this document, appropriately parameterized sponge or Sponge-F-style constructions could use these permutations with suitable capacities to target 128-bit or 256-bit generic security levels. Such modes would require their own domain separation, padding, rate/capacity choices, and security analysis, but the same WChain permutation design offers an efficient implementation base for those constructions.

4.4 Features

WChain is designed as a wide-state hash construction with a feedback-chain structure and a word-oriented permutation. Version I targets a compact 512-bit digest setting with a 9×64 state and 18 rounds. Version II targets a 1024-bit digest setting with a split $9 \times 64 + 9 \times 64$ state and 24 rounds. Its main features are summarized as follows.

- **Innovation.** WChain combines structural and component-level design ideas: the FChain feedback structure, the final checksum block Σ , a reusable wide permutation, transparent lane-rotated constants, a 9-bit NAND2/chi column S-box, and a sparse but invertible Mix Words layer. This reflects a design trend toward wide-state hash functions that combine explicit structural countermeasures with implementation-oriented permutation components. The theoretical novelty lies in treating the iteration structure, the message expansion, and the permutation diffusion as coupled design objects rather than independent modules.
- **Simplicity.** The round function is composed of a small number of easy-to-describe operations: XOR, rotations, a fixed 9-bit S-box, a fixed coordinate permutation, and a sparse word-mixing layer. This makes the algorithm easier to understand, implement, test, and independently analyze. A simple component structure is also useful for security evaluation, because differential, linear, invariant-subspace, integral, and implementation-level diffusion checks can be reproduced with compact models.
- **Flexibility.** The underlying 576-bit and 1152-bit permutations can support more than one mode of operation. The specification focuses on the FChain hash mode, but the same permutation core can be used as a basis for message authentication codes, XOF-like modes, or sponge/Sponge-F-style constructions after suitable domain separation, padding, rate/capacity choices, and mode-specific security analysis are defined. The word-oriented design also supports portable implementations across constrained devices, general-purpose CPUs, and high-throughput platforms.
- **Implementation efficiency.** WChain is designed to match modern platform features. The S-box layer is bit-sliced across columns, the Mix Words layer is a fixed XOR-and-rotation network, and the message expansion is intentionally lightweight. These properties support scalar optimization, SIMD batching with instruction-set acceleration such as AVX2 and AVX-512, and compact hardware implementations. On resource-constrained platforms, the same sparse operation set helps reduce memory traffic and control overhead.

The security claims in this specification are based on the analyses and experiments reported below: reduced-round trail searches, structural attack estimates, invariant-subspace checks, and implementation-oriented diffusion tests. Full differential-hull optimality and exhaustive full-round cryptanalysis are not claimed.

5 Security Analysis

This section summarizes the current evidence for the WChain specification. The results are divided by evidence type. Exact local results come from exhaustive component tables, rank computations, or exact low-weight enumeration. Search-based trail results are reported as found and verified single characteristics; they are not differential or linear hull bounds

and are not global optimality proofs. Generic structural estimates assume no dedicated shortcut for the compression function.

5.1 Generic Structural Attacks on FChain

In an idealized feed-forward compression model, the compression function

$$\text{CF}(V, B) = F_B(V) \oplus V \quad (38)$$

is expected to have birthday-bound type collision security over the hidden b -bit chaining state. For an adversary making at most q compression-function queries, the target collision-advantage estimate is

$$\text{Adv}_{\text{CF}}^{\text{coll}}(q) = O\left(\frac{q^2}{2^b}\right). \quad (39)$$

Since the final digest is truncated to h bits, the generic digest-collision bound is $2^{h/2}$, whereas a collision in the internal chaining state requires about $2^{b/2}$ work. For Version I these values are 2^{256} and 2^{288} , respectively; for Version II they are 2^{512} and 2^{576} .

The WChain iteration differs from plain Merkle–Damgård hashing in two structural ways. First, the chaining update contains a two-step feedback term,

$$H_i = \text{CF}(H_{i-1}, M_i) \oplus H_{i-2}. \quad (40)$$

Second, after all real message blocks have been processed, the construction applies a final checksum block

$$\Sigma = M_1 \oplus M_2 \oplus \cdots \oplus M_l, \quad H_{l+1} = \text{CF}(H_l, \Sigma). \quad (41)$$

Thus a final internal collision between two messages is obtained if both

$$H_l = H'_l \quad \text{and} \quad \Sigma = \Sigma' \quad (42)$$

hold. The checksum should therefore not be described as preventing multicollisions outright; it adds a checksum-collision constraint to the usual internal-state collision condition.

5.1.1 Length extension attack.

The current WChain specification uses 10^* padding. Let

$$M^* = M_1 \parallel \cdots \parallel M_l$$

be the padded message and

$$\Sigma = M_1 \oplus \cdots \oplus M_l.$$

After the real message blocks are processed, WChain applies a separate finalization call

$$H_{l+1} = \text{CF}(H_l, \Sigma), \quad Z = \text{Trunc}_h(H_{l+1}).$$

Thus the public digest is a truncated post-finalization value, not the chaining value H_l from which further message blocks can be processed.

Consider an attempted extension. Given $Z = \text{Hash}(M)$, an adversary may choose extra data T and form a longer syntactic message in which the delimiter and zero bits of M^* are treated as ordinary message bits. However, the digest of the longer message is defined by padding this new message again with 10^* , processing all resulting real message blocks, and using a new checksum

$$\Sigma' = M_1 \oplus \dots \oplus M_l \oplus T_1 \oplus \dots \oplus T_s \oplus \text{new padding block terms}.$$

The required computation therefore starts from the pre-finalization state H_l , not from the published Z , and it must end with the updated checksum Σ' . Continuing directly from Z is impossible because Z omits $b - h$ state bits and because it is already after the final compression call.

Enumerating only the missing digest bits gives 2^{b-h} candidates for H_{l+1} , namely 2^{64} candidates for Version I and 2^{128} for Version II. This enumeration is still not a complete length-extension attack: the attacker would additionally need a pre-finalization state and checksum relation consistent with $H_{l+1} = \text{CF}(H_l, \Sigma)$, then recompute the extension under Σ' . Therefore WChain does not provide the standard Merkle–Damgård $O(1)$ -cost extension interface. This is a generic structural argument about the current 10^* -padded construction; it does not replace dedicated analysis of the compression function.

5.1.2 Multicollisions and checksum correction.

Joux showed that a 2^r -multicollision for a standard iterated hash can be built with about $r2^{h/2}$ compression-function work [1]. For WChain, a direct adaptation must first build collisions in the hidden b -bit internal state. If $\mu + 1$ binary collision stages are built, all $2^{\mu+1}$ paths reach the same intermediate state at cost

$$C_\Sigma = (\mu + 1)2^{b/2}. \quad (43)$$

Each binary stage gives a checksum difference $\delta_i \in \mathbb{F}_2^\mu$ between its two branches. Since $\mu + 1$ vectors in a μ -dimensional binary vector space are linearly dependent, there is a nonempty set I such that

$$\bigoplus_{i \in I} \delta_i = 0. \quad (44)$$

Choosing the alternate branch exactly at the stages in I gives two different paths with the same intermediate state and the same final checksum, and hence a final collision by (42). For Version I,

$$C_\Sigma = 1153 \cdot 2^{288} \approx 2^{298.2}, \quad (45)$$

and for Version II,

$$C_\Sigma = 2305 \cdot 2^{576} \approx 2^{587.2}. \quad (46)$$

These values are above the generic digest birthday bounds 2^{256} and 2^{512} , respectively, so this constrained multicollision route is not a better generic final-digest collision attack.

If each collision stage offers multiple branch-difference candidates, the checksum condition can also be modeled as a generalized-birthday problem in the sense of Wagner [2]. With $k = 2^a$ lists and list size about $2^{\mu/(a+1)}$, a generic estimate is

$$C_{\text{GB}} \approx 2^{b/2+a+\mu/(a+1)}. \quad (47)$$

Without a shortcut for the internal compression-function collisions, this does not improve the linear-dependence estimate above. If a dedicated collision shortcut exists, the $2^{b/2}$ term in these estimates must be replaced by the actual internal collision cost.

5.1.3 Herding attacks.

The herding or Nostradamus attack builds a diamond of internal states and later bridges a chosen prefix into the committed value [3]. For a hidden b -bit state and a diamond-width parameter d , the generic cost is estimated as

$$C_{\text{herd}}(d) \approx 2^{(b+d)/2} + 2^{b-d} + C_{\Sigma}. \quad (48)$$

Taking $d \approx b/3$ gives

$$C_{\text{herd}} \approx 2^{2b/3} + C_{\Sigma}, \quad (49)$$

which is about $2^{384} + 2^{298.2}$ for Version I and $2^{768} + 2^{587.2}$ for Version II. The checksum adds a final XOR-sum matching condition, but it should be treated as an additional generalized-birthday constraint rather than as an absolute barrier.

5.1.4 Long-message second preimages.

Very long Merkle–Damgård messages admit second-preimage attacks below the brute-force preimage bound [4]. For a 2^k -block target message, replacing the visible MD state length by WChain’s hidden b -bit state and adding the checksum correction gives the generic estimate

$$C_{2\text{nd}}(k) \approx k2^{b/2+1} + 2^{b-k+1} + C_{\Sigma}. \quad (50)$$

For Version I this becomes

$$C_{2\text{nd}}(k) \approx k2^{289} + 2^{577-k} + 2^{298.2}, \quad (51)$$

and it reaches the $2^{298.2}$ checksum-correction scale only for extremely long messages, roughly $k \gtrsim 279$. For Version II,

$$C_{2\text{nd}}(k) \approx k2^{577} + 2^{1153-k} + 2^{587.2}, \quad (52)$$

with the analogous crossover around $k \gtrsim 566$.

The estimates above are generic. They do not rule out dedicated attacks on the compression function, the message expansion, or the internal permutation. Conversely, the checksum finalization should be interpreted as an additional constraint on generic structural attacks, not as a stand-alone proof of resistance.

5.2 Component Cryptanalysis

5.2.1 S-box Properties

The 9-bit NAND2/chi S-box is a permutation. Its forward coordinate degree is 2, its inverse coordinate degree is 5, its maximum DDT entry is 128, and its maximum absolute LAT entry is 128. Thus the maximum single-S-box differential probability is 2^{-2} , and the maximum single-S-box linear correlation is 2^{-1} .

5.2.2 Column Permutation $F_\pi = \pi \circ S$

For the column-level nonlinear-plus-wiring layer, define

$$F_\pi(a) = \pi(S(a)), \quad \pi = [0, 7, 5, 3, 1, 8, 6, 4, 2]. \quad (53)$$

Here π is an output-coordinate permutation on the 9-bit S-box result. Therefore F_π is also a 9-bit permutation, and its DDT and LAT are only coordinate-reindexed versions of the S-box DDT and LAT. More explicitly, for an input difference α and an output difference β ,

$$\text{DDT}_{F_\pi}[\alpha, \beta] = \text{DDT}_S[\alpha, \pi^{-1}(\beta)],$$

and the analogous LAT relation permutes the output mask. Consequently, F_π keeps the same bijectivity, algebraic degree, inverse degree, maximum differential probability, and maximum linear correlation as S .

The security role of π is therefore not to improve the local S-box probability table. Its role is to change the coordinate alignment between the S-box output and the sparse L_{5w} input support. Without this wiring step, low-weight S-box outputs remain more tightly aligned with the same lane positions entering the Mix Words layer, and the influence is easier to keep concentrated in a small set of coordinates. With the selected π , the checked two-step propagation

$$L_{5w} \circ \pi \circ L_{5w} \circ \pi(e_i)$$

activates 25 bit positions in 25 distinct z -columns for every single-bit input e_i . In the full reachable-support propagation through $S/\pi/L_{5w}$, a single active bit covers all nine lanes after two rounds, all 64 z -columns after three rounds, and all 576 state bits after four rounds. Thus π is a low-cost diffusion-coupling layer: it preserves the local S-box security metrics while improving how local S-box activity is presented to the word-mixing layer.

5.2.3 Mix Words Layer Cryptanalysis

This subsection analyzes the Mix Words layers as linear components of the adopted WChain permutations. The Version I object is the 9×64 half-layer L_{5w} . The Version II object is the full split layer L_{5w}^{II} , including CrossSwap5 and the asymmetric right-half Mix Words table.

For Version I, L_{5w} is sparse in the forward direction: every output bit depends on five input bits. The cryptanalytic concern for such a lightweight linear layer is that low-weight differences or linear masks might pass through with too little expansion, or might form small iterative cores. Exact low-weight enumeration gives 1/2/3-bit branch numbers 6/8/12 in both the differential direction L_{5w} and the transpose-linear direction L_{5w}^T . A targeted 4-bit search found best branch number 14 and did not find a 4-to-4 iterative core. The verified properties are summarized in Table 6.

Table 6: Verified properties of L_{5w} for Version I

Property	Value
$\text{rank}(L_{5w})$	576
$\text{rank}(L_{5w}^T)$	576
Row/column weight	{5 : 576}
Number of ones in L_{5w}	2880
Number of ones in L_{5w}^{-1}	164416
Inverse bit weight, min/avg/max	269/285.44/305
Inverse lane coverage	{9 : 576}
Inverse z -column coverage	{64 : 576}
1-bit / 2-bit / 3-bit BN of L_{5w}	6/8/12
1-bit / 2-bit / 3-bit BN of L_{5w}^T	6/8/12
Targeted 4-bit search	best BN 14; no 4-to-4 core found

Diffusion effect. For every single-lane input, the concrete two-step propagation

$$L_{5w} \circ \pi \circ L_{5w} \circ \pi(e_i) \quad (54)$$

activates exactly 25 bit positions in 25 distinct z columns. In reachable-support propagation through $S/\pi/L_{5w}$, a single active bit covers all nine lanes after two rounds, all 64 z columns after three rounds, and all 576 state bits after four rounds.

For Version II, the full split layer remains invertible because CrossSwap5 is a word permutation and because each half-layer is invertible. The split layer has row and column weight five over the 1152-bit state, and the half-layer 1/2/3-bit branch numbers remain 6/8/12 for both forward and transpose propagation. The verified split-layer properties are summarized in Table 7, and the inverse dependency coverage is shown in Table 8.

The inverse-coverage figures are properties of the linear Mix Words Layer only. They describe how many input positions can influence a single output bit when the layer is inverted; they should not be read as differential probabilities through the nonlinear S-box layer.

The Version II split layer was also checked for left/right self-differential relations. For the relation

$$R[i] = \text{rotr}_{64}(L[i], k), \quad 0 \leq i < 9, \quad (55)$$

Table 7: Verified properties of the Version II split linear layer

Property	Value
State size	$18 \times 64 = 1152$ bits
Rank of L_{5w}^{II} and $(L_{5w}^{\text{II}})^T$	1152
Row/column weight	$\{5 : 1152\}$
Number of ones in L_{5w}^{II}	5760
Number of ones in $(L_{5w}^{\text{II}})^{-1}$	328832
Inverse z -column coverage	$\{64 : 1152\}$
1-bit / 2-bit / 3-bit BN of each half	6/8/12
1-bit / 2-bit / 3-bit BN of each transpose half	6/8/12
Single-bit reachable full-word coverage	2 rounds
Single-bit reachable full- z coverage	3 rounds
Single-bit reachable full-bit coverage	4 rounds

the adopted asymmetric split layer does not preserve a closed 576-dimensional subspace for any shift $k \in \mathbb{Z}_{64}$. The minimum defect rank over all k is 566 and the maximum kernel dimension is 10. This follows the same structural warning used in Keccak self-symmetry distinguishers [5]: repeated or mirrored S-box events should not be counted as independent when the state relation is preserved. The result is a component-level structural check for the current Mix Words layer.

5.2.4 Full-Diffusion Check of the Permutation Core

We also check the full-diffusion behavior of the permutation core under a single-bit input difference. The forward direction propagates reachable supports through $S/\pi/L_{5w}$, and the backward direction propagates a single-bit output difference through $L_{5w}^{-1}/\pi^{-1}/S^{-1}$. Round constants and message injections are XOR constants and therefore do not affect this support propagation. The numbers in Tables 9 and 10 are worst-case minima over all single-bit starting positions; they measure reachable support only and should not be interpreted as single-trail probabilities.

For Version I, the forward permutation core reaches all z -columns after three rounds and all 576 state bits after four rounds, while the inverse direction reaches full-state support after one inverse round because L_{5w}^{-1} is dense. For Version II, the split permutation reaches all 128 split S-box columns and all physical z -columns after three forward rounds, and all 1152 state bits after four forward rounds. In the inverse direction, one round already covers all 18 words and all physical z -columns, and two inverse rounds cover the full 1152-bit state.

Table 8: Inverse coverage of the Version II split Mix Words Layer

Property	Value
$(L_{5w}^{\text{II}})(L_{5w}^{\text{II}})^{-1} = I$	verified
Ones in left/right half inverses	164416/164416
Ones in full inverse	328832
Full inverse row-weight distribution	{271 : 128, 277 : 128, 279 : 256, 283 : 128, 287 : 128, 289 : 128, 299 : 128, 305 : 128}
Full inverse column-weight distribution	{269 : 128, 271 : 128, 275 : 128, 277 : 128, 279 : 128, 297 : 256, 299 : 128, 305 : 128}
Single-bit inverse bit weight, min/avg/max	269/285.44/305
Single-bit inverse word coverage	{9 : 1152}
Single-bit inverse split S-box-column coverage	min/avg/max 119/121.67/126; histogram {119 : 256, 120 : 128, 121 : 128, 122 : 256, 123 : 256, 126 : 128}
Single-bit inverse physical z -column coverage	{64 : 1152}

Table 9: Version I single-bit reachable-support diffusion

Direction	Round	Active bits	Active lanes	Active z -columns
Forward	1	15	8	12
Forward	2	184	9	60
Forward	3	574	9	64
Forward	4	576	9	64
Backward	1	576	9	64
Backward	2	576	9	64
Backward	3	576	9	64
Backward	4	576	9	64

5.3 Differential and Linear Characteristics of Permutation

This subsection analyzes the state permutation used inside the compression function, rather than the full message-dependent compression function. The message-injection blocks and round constants are fixed and equal in the paired computations; therefore they cancel in XOR-difference propagation and do not affect linear-mask propagation. The reduced-round searches are consequently performed on the state core $\text{Round} = L_{5w} \circ \pi \circ S$ for Version I and on the corresponding split state core for Version II. The results do not model the full 18-round Version I hash or the full 24-round Version II hash as probability-bearing hulls.

The numerical characteristics in the following tables are generated by a table-driven trail search and then verified by exact local transition tables. This first search layer is used to find good candidate trails efficiently. For differential trails, each active 9-bit S-box

Table 10: Version II single-bit reachable-support diffusion

Direction	Round	Active bits	Active words	Active S-box columns	Active z -columns
Forward	1	15	12	14	13
Forward	2	199	18	105	62
Forward	3	1068	18	128	64
Forward	4	1152	18	128	64
Backward	1	1071	18	119	64
Backward	2	1152	18	128	64
Backward	3	1152	18	128	64
Backward	4	1152	18	128	64

transition $\alpha \mapsto \beta$ is selected from the full DDT and assigned the local cost

$$-\log_2(\text{DDT}[\alpha, \beta]/512).$$

Candidate trails are expanded by S-box columns, ranked by accumulated S-box cost and support size, and then propagated exactly through π and the Mix Words layer. For Version II, the exact propagation additionally includes π_{split} , CrossSwap_5 , and L_{5w}^{II} . Each reported differential characteristic was verified by replaying all local DDT transitions and the exact linear layer.

For linear characteristics, the search uses the dual propagation model. Masks are propagated backward through the transpose Mix Words layers and inverse wiring, and each local S-box mask transition is checked against the full LAT. For a local mask transition $\alpha \mapsto \beta$, the accumulated correlation cost is

$$-\log_2(|\text{LAT}[\alpha, \beta]|/512),$$

and the squared-correlation exponent is twice the accumulated correlation exponent. The reported linear rows are therefore valid found single linear characteristics obtained under the transpose-linear propagation model.

To make the optimality question explicit, we also formulate an exact large-S-box MILP model for the same state permutation. In this model each column S-box is treated as one 9-bit S-box transition rather than decomposed into coordinate equations. For differential search, the variables $(\Delta X_{r,z}, \Delta Y_{r,z}) \in \mathbb{F}_2^9 \times \mathbb{F}_2^9$ are constrained by the exact DDT support; for linear search, the corresponding mask variables are constrained by the exact LAT support. The deterministic layers are encoded exactly as XOR equations over $\mathbb{F}_2$. Thus

$$\Delta X_{r+1} = (L_{5w} \circ \pi)(\Delta Y_r)$$

is used for differential propagation in Version I, while linear masks use the transpose equation

$$Y_r = (L_{5w} \circ \pi)^T X_{r+1}.$$

For Version II the same construction is applied to the split deterministic layer $(L_L, L_R) \circ \text{CrossSwap}_5 \circ \pi_{\text{split}}$. The objective is the sum of the local DDT probability costs or LAT correlation costs over all active S-box columns.

The implemented MILP uses a single 9-bit transition table for each S-box instance, together with lazy exact-table cuts for DDT/LAT legality and local weight. This avoids introducing one binary selector for every nonzero DDT or LAT entry in every S-box instance, while keeping the linear layer exact. A MILP result is considered an optimal characteristic only when the solver terminates with an optimality certificate. Unless such a certificate is explicitly stated, the tables below should be read as verified found single differential and linear characteristics, not as MILP/SAT-certified optima and not as differential, linear, or differential-linear hull probabilities.

Table 11: Best found verified differential characteristics for Version I

Rounds	Exponent	Active S-boxes	Output bits	Output columns	Probability
1	2	1	5	5	2^{-2}
2	12	6	25	25	2^{-12}
3	62	31	105	59	2^{-62}
4	240	90	206	64	2^{-240}
5	520	154	248	64	2^{-520}
6	851	218	241	63	2^{-851}
7	1174	281	256	64	2^{-1174}

Table 12: Best found verified differential characteristics for Version II

Rounds	Exponent	Active S-boxes	Output bits	Output columns	Probability
1	2	1	5	5	2^{-2}
2	12	6	25	22	2^{-12}
3	61	28	106	76	2^{-61}
4	258	104	330	121	2^{-258}
5	768	226	525	127	2^{-768}
6	1444	353	480	126	2^{-1444}
7	2087	479	476	125	2^{-2087}

All entries in the Version I differential table (Table 11), the Version II differential table (Table 12), and the linear table (Table 13) should be interpreted as verified found single characteristics. In particular, for Version I we found a verified 5-round differential characteristic with exponent 520, i.e., probability 2^{-520} . For Version II we found a verified 5-round differential characteristic with exponent 768, i.e., probability 2^{-768} .

These permutation trails also give a coarse heuristic for reduced-round compression-function collisions. A WChain message block contains $\mu = 2b$ bits of message freedom, namely 1152 bits for Version I and 2304 bits for Version II. Comparing this freedom with

Table 13: Best found verified backward linear characteristics

Version	Backward rounds	Corr. exp.	Squared-corr. exp.	Active S-boxes
I	1	5	10	5
I	2	29	58	24
I	3	107	214	73
I	4	254	508	137
II	1	5	10	5
II	2	30	60	28
II	3	132	264	106
II	4	389	778	228

the found 7-round forward differential characteristics gives a rough collision-search scale: Version I has a 7-round single-trail exponent 1174, close to its 1152-bit message freedom, while Version II has a 7-round exponent 2087, below its 2304-bit message freedom. This suggests that an 8-round reduced compression-collision search may be plausible in this heuristic model for both versions. However, this is only an estimate. A real compression collision must also satisfy the feed-forward cancellation condition, so the head and tail differences have to be compatible and cancel with the final round-message difference. These endpoint constraints may consume additional degrees of freedom, and no concrete 8-round compression collision is claimed from the single-trail data alone.

5.4 Meet-in-the-Middle Preimage Attack on the Compression Function

This subsection considers reduced-round preimage attacks on the WChain compression function only. For a fixed chaining value V , the reduced-round compression function is

$$\text{CF}(V, B) = X_R \oplus \widehat{B}_R \oplus V, \quad (56)$$

where V is the chaining value, B is parsed into $\widehat{B}_0, \widehat{B}_1$, the message expansion computes $\widehat{B}_j$, and X_R is the state after R message-injected rounds. The full parameters are $b = 576, R = 18$ for Version I and $b = 1152, R = 24$ for Version II. The concrete screens below are reduced-round diagnostics and should not be read as full-round attacks.

Diffusion-based cryptanalysis estimate. We estimate the useful MitM window in the same spirit as PM/IS-style estimates for SPN primitives. In a keyed block-cipher setting, the inserted key differences provide the degrees of freedom; in the WChain compression function, the analogous freedom comes from the two message seeds $\widehat{B}_0, \widehat{B}_1$. Since one compression message block has length $2b$, these two seeds provide $2b$ bits of message freedom that may be used to build forward and backward computations.

The diffusion checks in Section 5.2 show that, for Version I, a single-bit forward difference reaches all 64 S-box columns after three rounds and reaches complete bit support

after four rounds. In the inverse direction, one round already gives full lane and z -column support. Thus, if partial matching (PM) is counted at the S-box-column level, the number of rounds over which forward and backward non-linear dependencies can remain separated is upper bounded by

$$r_{\text{PM}} \leq (3 - 1) + (1 - 1) + 1 = 3.$$

If complete bit support is required on the forward side, the conservative bound is $r_{\text{PM}} \leq (4 - 1) + (1 - 1) + 1 = 4$.

The initial-structure (IS) condition is that message-difference trails generated from the forward side and those generated from the backward side do not share active non-linear S-box components. Because the forward trail reaches all S-box columns after three rounds and the backward trail has full inverse support after one round, a long IS region is not expected. We therefore reserve one round for the initial message/state split. If a splice-and-cut step can add one extra round on each side, the resulting reduced-round MitM preimage window is estimated as

$$3 + 1 + 4 = 8.$$

This 8-round estimate is a diffusion-based upper-bound model for reduced-round

5.5 Zero-Sum Attacks

The zero-sum discussion is based on the algebraic degree of the fixed-message permutation core. The experimental integral and division-property screens performed for this specification follow the same growth pattern as the local S-box degree: the forward 9-bit S-box has coordinate degree 2, while the inverse S-box has coordinate degree 5. Since π , the Mix Words layer, CrossSwap5, and the round-constant layer are affine, they do not increase the algebraic degree. We therefore summarize the zero-sum evidence by the direct degree estimates

$$D_r^+ \leq \min(N, 2^r), \quad D_r^- \leq \min(N, 5^r),$$

where N is the state size, D_r^+ is the forward degree bound after r rounds, and D_r^- is the corresponding inverse-round bound.

For an m -dimensional active affine subspace, every output coordinate is balanced whenever its degree is strictly smaller than m . Thus a whole-state zero-sum certificate is obtained while the bound remains below the full state size. Table 14 gives the resulting estimates for the current two versions.

In this coarse estimate, the 9-round forward Version I bound corresponds to a degree scale of 512, often reported as a data scale around 2^{512} ; the next forward round already saturates the 576-bit state bound. Version II has the same forward degree growth but a larger 1152-bit state, so the degree-bound certificate extends to 10 forward rounds. In the inverse direction the degree grows according to 5^r , so the useful reduced-round window is shorter: three inverse rounds for Version I and four inverse rounds for Version II.

Table 14: Zero-sum estimates from algebraic-degree growth of the permutation core

Version	State size N	Direction	Last certified round	Saturation boundary
I	576	Forward, degree 2^r	9 rounds, $D_9^+ \leq 512$	10 rounds, $D_{10}^+ = 576$
I	576	Backward, degree 5^r	3 rounds, $D_3^- \leq 125$	4 rounds, $D_4^- = 576$
II	1152	Forward, degree 2^r	10 rounds, $D_{10}^+ \leq 1024$	11 rounds, $D_{11}^+ = 1152$
II	1152	Backward, degree 5^r	4 rounds, $D_4^- \leq 625$	5 rounds, $D_5^- = 1152$

5.6 Round Constants and Invariant Structures

The round constants follow the invariant-aware design criterion of Wei–Ye–Wu–Pasalic [11]. The adopted lane-rotated constants

$$X[i] \leftarrow X[i] \oplus \text{rotr}_{64}(\pi_j^{(16)}, i)$$

for Version I, and the corresponding split-lane rule for Version II, are intended to avoid low-dimensional identical-lane constant subspaces. Current low-degree checks found no degree-at-most-two lifted invariant failures for the adopted constants. This is evidence against the tested invariant-subspace patterns, not a proof against all generalized nonlinear invariant attacks.

Version II was also checked for Keccak-style left/right self-symmetry relations [5]. The asymmetric right-half rotation offsets remove the full closed relation that appears in the symmetric split control case. No shift $k \in \mathbb{Z}_{64}$ gives a closed 576-dimensional left/right self-differential subspace for the adopted Version II Mix Words layer.

6 Implementation and Performance

6.1 Reference Implementation

The reference implementation is intended to be the unambiguous executable specification of both parameter sets. It uses big-endian 64-bit lane encoding at the external byte interface, all-zero initial values, 10* padding without an encoded length field, the FChain feedback rule, the Feistel-type message expansion, the final Σ block, and leftmost digest truncation. The conforming round counts are $R = 18$ for Version I and $R = 24$ for Version II. The public C API is declared in `c/wchain_c.h`:

```
wchain_v1_hash_c(msg,msg_len,out[64])
wchain_v2_hash_c(msg,msg_len,out[128])
wchain_v1_hash_bits_c(msg,msg_len_bits,out[64])
wchain_v2_hash_bits_c(msg,msg_len_bits,out[128])
wchain_crypt_hash_c(digest_len_bits,msg,msg_len_bits,out)
```

The reference source is used to regenerate KAT files and to check the optimized submission code. In particular, the optimized Version I and Version II sources were synchronized with

the current specification: 10^* padding is used in the hash layer, the message expansion produces $\widehat{B}_R$ for the feed-forward, and the KAT outputs are regenerated rather than inherited from the earlier round-count variants.

Table 15: Implementation artifacts

Artifact	Role	Notes
<code>c/wchain.c.c</code>	Executable reference	Complete V1/V2 padding, FChain, message expansion, compression, final Σ block, digest truncation, and KAT generation support.
<code>Implementations/Reference_Implementation/</code>	Submission reference code	CryptHash-style V1/V2 reference implementation with $R = 18/24$ and 10^* padding.
<code>Implementations/Optimized_Implementation/WChain-V1-512/</code>	Optimized V1/V2 baseline	Scalar optimized code with AVX2/AVX-512 compression helpers and regenerated KAT files.
<code>Implementations/Optimized_Implementation/WChain-V1-512-avxopt/</code>	V1 AVX optimization branch	Dedicated Version I scalar, AVX2 4-way, and AVX-512 8-way benchmark path.
<code>bench/run_x86_self_eval.sh</code>	x86 self-evaluation	Builds KAT, padding, API, memory, and performance checks for the submission layout.

6.2 Optimized Scalar Implementation

The optimized scalar implementation keeps one 64-bit machine word per WChain lane. The 9-bit S-box is evaluated in bit-sliced form over the 64 parallel z -columns, so one Boolean expression updates a full lane word. The intra-column permutation π and the Mix Words layer are fused into fixed straight-line rotate-XOR expressions, avoiding temporary column arrays and loop-carried indexing. Round-constant injection and round-message injection are also folded into the round input preparation.

For compression, the Feistel-type message expansion is implemented with a rolling three-buffer schedule. The code does not materialize the full $\widehat{B}_0, \dots, \widehat{B}_R$ array; instead it keeps the two previous expanded blocks, applies the lightweight MEF, and retains only the final $\widehat{B}_R$ needed by the feed-forward $X_R \oplus \widehat{B}_R \oplus V$. This reduces stack traffic and removes a visible source of overhead in the earlier implementation, where message injection and feed-forward copying were measured as separate hot spots.

6.3 SIMD Batch Implementation

The SIMD implementation is designed for throughput rather than single-message latency. AVX2 processes four independent messages in parallel and AVX-512 processes eight independent messages in parallel. In both cases, vector lane i stores the same WChain word

position from different messages. This layout matches the bit-sliced S-box, lane rotations, message expansion, and feed-forward operations without transposing the internal 9×64 state of a single message.

The AVX2 path uses vector shifts and ORs for 64-bit rotations and vector XORs for the Mix Words layer. The AVX-512 path additionally uses ternary Boolean instructions such as `vpternlogq` to combine the S-box Boolean formula and XOR trees more tightly. The AVX-512 implementation therefore reduces the number of issued logical instructions, although actual speedup still depends on the processor’s AVX-512 frequency behavior. The benchmark reports below are aggregate batch throughput; they should not be interpreted as single-message latency.

6.4 Version I 18-Round Performance Measurements

After changing Version I to $R = 18$, the reference implementation and the optimized scalar, AVX2, and AVX-512 paths were rebuilt and their KAT/self-tests were regenerated. The measurements in this subsection were obtained on the 224server1G x86 server with an Intel Xeon Platinum 8358 CPU and GCC 11.4.0. The previous 20-round AVX2 and AVX-512 numbers are not reused for the 18-round specification. A Version I compression block absorbs 1152 bits (144 bytes). Table 16 reports the core permutation and compression measurements.

Table 16: Optimized Version I 18-round permutation and compression performance on x86

Target		Batch size	Average ns	Throughput MB/s
18-round scalar	permutation,	1	311.27	–
18-round scalar	compression,	1	366.95	392.42
18-round AVX2	compression,	4	615.64	935.62
18-round AVX-512	compression,	8	826.50	1393.82

The scalar compression path is about $1.18\times$ the cost of the bare 18-round permutation in this x86 run. The additional time comes from block decoding, eighteen message injections, seventeen lightweight MEF calls, the final feed-forward, and output serialization. Relative to scalar compression, the AVX2 four-way path reaches about $2.38\times$ higher aggregate throughput, and the AVX-512 eight-way path reaches about $3.55\times$ higher aggregate throughput.

The full-hash numbers include padding, FChain iteration, the final Σ compression, and digest output. For large inputs, Version I reaches 332.66 MB/s in the optimized scalar path, 912.18 MB/s in the AVX2 four-way path, and 1339.94 MB/s in the AVX-512 eight-

Table 17: Optimized Version I 18-round full-hash throughput on x86

Input bytes	Scalar MB/s	AVX2 4-way MB/s	AVX-512 8-way MB/s
32	36.14	100.61	149.03
128	169.31	402.34	595.28
512	237.58	648.35	955.30
1024	262.12	694.29	1014.52
4096	302.23	835.26	1124.52
8192	317.71	876.01	1297.50
16384	326.46	888.56	1227.08
65536	332.66	912.18	1339.94

way path. The short-message rows show the expected fixed-cost effect of the finalization block and are therefore lower than the long-message steady-state throughput.

6.5 Version II Implementation Status

Version II uses the same coding discipline but a larger split state: $(9 \times 64)_L \parallel (9 \times 64)_R$, $R = 24$, 24 round-message injections, 23 MEF calls, CrossSwap5, and asymmetric left/right Mix Words tables. Since Version II has twice as many 64-bit words and four additional rounds, its per-compression latency is expected to be higher than Version I. Final Version II throughput should be reported from the current $R = 24$, 10*-padding benchmark harness.

6.6 Implementation Summary

The implementation strategy separates correctness and speed. The reference code is the executable specification and the KAT source. The optimized scalar code improves single-message latency by fusing the nonlinear layer, coordinate permutation, Mix Words layer, message injection, and rolling message expansion. The AVX2 and AVX-512 paths improve batch throughput by processing independent messages in parallel. Under the current x86 measurements, the optimized Version I path reaches 392.42 MB/s per compression call in scalar code, 935.62 MB/s with AVX2 4-way compression, and 1.39 GB/s with AVX-512 8-way compression.

WChain is also intended to be hardware-friendly. The 9-bit S-box is local to each column and can be instantiated in parallel, while the intra-column permutation, word rotations, XOR-based Mix Words layer, message injection, and feed-forward operations are composed of wiring and simple Boolean operations. These properties make the permutation and the compression function natural candidates for FPGA and ASIC implementations with low control complexity and high parallelism. A complete hardware evaluation, including area, critical path, throughput, and energy figures, remains future work and should be developed as a separate implementation study.

7 Statistical Randomness Tests

This section reports statistical randomness tests for WChain hash outputs. The tests were run on three input classes: high-density data, low-density data, and random data. For each test item and each input class, 1000 samples were evaluated. The significance level is $\alpha = 0.01$, and an item is considered passed when at least 981 out of 1000 samples pass the corresponding test. Tables 18 and 19 summarize the observed pass counts.

These tests are intended as sanity checks for the statistical behavior of the hash outputs. They do not replace cryptographic security analysis and should not be interpreted as proofs of collision resistance, preimage resistance, or resistance against distinguishers.

References

- [1] Antoine Joux. Multicollisions in iterated hash functions. application to cascaded constructions. In Matthew K. Franklin, editor, *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*, volume 3152 of *Lecture Notes in Computer Science*, pages 306–316. Springer, 2004.
- [2] David A. Wagner. A generalized birthday problem. In Moti Yung, editor, *Advances in Cryptology - CRYPTO 2002, 22nd Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 2002, Proceedings*, volume 2442 of *Lecture Notes in Computer Science*, pages 288–303. Springer, 2002.
- [3] John Kelsey and Tadayoshi Kohno. Herding hash functions and the nostradamus attack. In Serge Vaudenay, editor, *Advances in Cryptology - EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006, Proceedings*, volume 4004 of *Lecture Notes in Computer Science*, pages 183–200. Springer, 2006.
- [4] John Kelsey and Bruce Schneier. Second preimages on n -bit hash functions for much less than 2^n work. In Ronald Cramer, editor, *Advances in Cryptology - EUROCRYPT 2005, 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005, Proceedings*, volume 3494 of *Lecture Notes in Computer Science*, pages 474–490. Springer, 2005.
- [5] Sukhendu Kuila, Dhiman Saha, Madhumangal Pal, and Dipanwita Roy Chowdhury. Practical distinguishers against 6-round keccak-f exploiting self-symmetry. In David Pointcheval and Damien Vergnaud, editors, *Progress in Cryptology - AFRICACRYPT 2014 - 7th International Conference on Cryptology in Africa, Marrakesh, Morocco, May 28-30, 2014. Proceedings*, volume 8469 of *Lecture Notes in Computer Science*, pages 88–108. Springer, 2014.

Table 18: WCHAIN_V1 output randomness-test results, $\alpha = 0.01$, pass threshold 981/1000

Test item	High-density data	Low-density data	Random data
Monobit frequency test	990/1000 (pass)	990/1000 (pass)	989/1000 (pass)
Block frequency test ($m = 10000$)	994/1000 (pass)	995/1000 (pass)	987/1000 (pass)
Poker test ($m = 4$)	989/1000 (pass)	986/1000 (pass)	990/1000 (pass)
Poker test ($m = 8$)	988/1000 (pass)	992/1000 (pass)	987/1000 (pass)
Overlapping subsequence test ($m = 3$, P1)	996/1000 (pass)	986/1000 (pass)	999/1000 (pass)
Overlapping subsequence test ($m = 3$, P2)	996/1000 (pass)	984/1000 (pass)	995/1000 (pass)
Overlapping subsequence test ($m = 5$, P1)	992/1000 (pass)	992/1000 (pass)	991/1000 (pass)
Overlapping subsequence test ($m = 5$, P2)	987/1000 (pass)	990/1000 (pass)	987/1000 (pass)
Runs total test	992/1000 (pass)	989/1000 (pass)	994/1000 (pass)
Runs distribution test	992/1000 (pass)	990/1000 (pass)	991/1000 (pass)
Longest run of ones in a block ($m = 10000$)	994/1000 (pass)	989/1000 (pass)	990/1000 (pass)
Longest run of zeros in a block ($m = 10000$)	995/1000 (pass)	993/1000 (pass)	983/1000 (pass)
Binary derivative test ($k = 3$)	988/1000 (pass)	988/1000 (pass)	987/1000 (pass)
Binary derivative test ($k = 7$)	990/1000 (pass)	989/1000 (pass)	987/1000 (pass)
Autocorrelation test ($d = 1$)	992/1000 (pass)	989/1000 (pass)	994/1000 (pass)
Autocorrelation test ($d = 2$)	994/1000 (pass)	987/1000 (pass)	988/1000 (pass)
Autocorrelation test ($d = 8$)	992/1000 (pass)	989/1000 (pass)	993/1000 (pass)
Autocorrelation test ($d = 16$)	983/1000 (pass)	993/1000 (pass)	993/1000 (pass)
Matrix rank test	991/1000 (pass)	992/1000 (pass)	987/1000 (pass)
Cumulative sums test (forward)	989/1000 (pass)	992/1000 (pass)	991/1000 (pass)
Cumulative sums test (backward)	992/1000 (pass)	988/1000 (pass)	991/1000 (pass)
Approximate entropy test ($m = 2$)	997/1000 (pass)	986/1000 (pass)	999/1000 (pass)
Approximate entropy test ($m = 5$)	992/1000 (pass)	995/1000 (pass)	993/1000 (pass)
Linear complexity test ($m = 500$)	990/1000 (pass)	990/1000 (pass)	995/1000 (pass)
Linear complexity test ($m = 1000$)	989/1000 (pass)	988/1000 (pass)	992/1000 (pass)
Maurer's universal statistical test ($L = 7, Q = 1280$)	988/1000 (pass)	985/1000 (pass)	989/1000 (pass)
Discrete Fourier transform test	987/1000 (pass)	990/1000 (pass)	990/1000 (pass)

Table 19: WCHAIN_V2 output randomness-test results, $\alpha = 0.01$, pass threshold 981/1000

Test item	High-density data	Low-density data	Random data
Monobit frequency test	987/1000 (pass)	991/1000 (pass)	995/1000 (pass)
Block frequency test ($m = 10000$)	993/1000 (pass)	991/1000 (pass)	988/1000 (pass)
Poker test ($m = 4$)	983/1000 (pass)	990/1000 (pass)	991/1000 (pass)
Poker test ($m = 8$)	989/1000 (pass)	986/1000 (pass)	991/1000 (pass)
Overlapping subsequence test ($m = 3$, P1)	989/1000 (pass)	988/1000 (pass)	987/1000 (pass)
Overlapping subsequence test ($m = 3$, P2)	988/1000 (pass)	991/1000 (pass)	992/1000 (pass)
Overlapping subsequence test ($m = 5$, P1)	989/1000 (pass)	989/1000 (pass)	988/1000 (pass)
Overlapping subsequence test ($m = 5$, P2)	993/1000 (pass)	990/1000 (pass)	992/1000 (pass)
Runs total test	992/1000 (pass)	990/1000 (pass)	983/1000 (pass)
Runs distribution test	994/1000 (pass)	987/1000 (pass)	993/1000 (pass)
Longest run of ones in a block ($m = 10000$)	991/1000 (pass)	991/1000 (pass)	991/1000 (pass)
Longest run of zeros in a block ($m = 10000$)	990/1000 (pass)	991/1000 (pass)	989/1000 (pass)
Binary derivative test ($k = 3$)	992/1000 (pass)	992/1000 (pass)	990/1000 (pass)
Binary derivative test ($k = 7$)	995/1000 (pass)	988/1000 (pass)	988/1000 (pass)
Autocorrelation test ($d = 1$)	992/1000 (pass)	990/1000 (pass)	983/1000 (pass)
Autocorrelation test ($d = 2$)	983/1000 (pass)	990/1000 (pass)	986/1000 (pass)
Autocorrelation test ($d = 8$)	991/1000 (pass)	987/1000 (pass)	992/1000 (pass)
Autocorrelation test ($d = 16$)	990/1000 (pass)	987/1000 (pass)	991/1000 (pass)
Matrix rank test	992/1000 (pass)	999/1000 (pass)	988/1000 (pass)
Cumulative sums test (forward)	988/1000 (pass)	990/1000 (pass)	995/1000 (pass)
Cumulative sums test (backward)	984/1000 (pass)	990/1000 (pass)	992/1000 (pass)
Approximate entropy test ($m = 2$)	989/1000 (pass)	988/1000 (pass)	987/1000 (pass)
Approximate entropy test ($m = 5$)	991/1000 (pass)	988/1000 (pass)	990/1000 (pass)
Linear complexity test ($m = 500$)	996/1000 (pass)	992/1000 (pass)	992/1000 (pass)
Linear complexity test ($m = 1000$)	991/1000 (pass)	987/1000 (pass)	985/1000 (pass)
Maurer's universal statistical test ($L = 7, Q = 1280$)	992/1000 (pass)	990/1000 (pass)	991/1000 (pass)
Discrete Fourier transform test	986/1000 (pass)	991/1000 (pass)	990/1000 (pass)

- [6] Zhenzhen Bao, Xiaoyang Dong, Jian Guo, Zheng Li, Danping Shi, Siwei Sun, and Xiaoyun Wang. Automatic search of meet-in-the-middle preimage attacks on AES-like hashing. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 771–804. Springer, 2021. IACR Cryptology ePrint Archive, Report 2020/467.
- [7] Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678, 2016.
- [8] Muhammad ElSheikh and Amr M. Youssef. On MILP-based automatic search for bit-based division property for ciphers with (large) linear layers. Cryptology ePrint Archive, Paper 2021/643, 2021. Published at ACISP 2021.
- [9] Jian Guo, Guozhen Liu, Ling Song, and Yi Tu. Exploring SAT for cryptanalysis: (quantum) collision attacks against 6-round SHA-3. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part III*, volume 13793 of *Lecture Notes in Computer Science*, pages 645–674. Springer, 2022.
- [10] Boxin Zhao, Qingliang Hou, Lingyue Qin, and Xiaoyang Dong. Triangulating meet-in-the-middle attack. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part V*, volume 16004 of *Lecture Notes in Computer Science*, pages 260–292. Springer, 2025.
- [11] Yongzhuang Wei, Tao Ye, Wenling Wu, and Enes Pasalic. Generalized nonlinear invariant attack and a new design criterion for round constants. *IACR Trans. Symmetric Cryptol.*, 2018(4):62–79, 2018.